DIATraceExtractor V0.5.0: release report (2026-10-20)

Versions. This report accompanies release **V0.5.0** of `DIATraceExtractor` (okohlbacher/dia-trace-extract-prototype). Until V0.5.0 the extraction had not changed since V0.2 (commit 663dc39 = cb466f0 + the report); V0.2.1–V0.2.7 added the `-scheme B` preset, macOS and Windows support, three defect fixes, a faster mzML ingest (V0.2.7: positional reads and an unbounded in-order commit, output byte-identical) and this report's corrections; V0.2.8–V0.2.14 changed no source file, only the Windows build, the install rules and the release workflow, and added binary packages for five platforms with checksums and provenance attestations (Appendix A). V0.2.15 reworked the command line after an adversarial review: defaults that need no tuning (RT map, RT window, tolerances measured from the run; all physical cores), one Parquet file per run, consistent option names and 13 basic options (Appendix A). V0.2.16: the physical cores by default, the RT-window multiple chosen on all benchmark data (7σ + 2 FWHM, §2), the 0.2.14 option names accepted again, the peptide-decoy experiment (§5), a macOS installer package (Appendix A). V0.3.0: instrument defaults as data files with `-instrument`, a shorter log, a one-line message for a mistyped option, `-write_ini` on the current names (Appendix A, `CHANGELOG.md`), and a fix: since V0.2.15, where the measured RT window shrinks faster than the RT grows, the extraction wrote past the end of some traces (timsTOF A01; see below). V0.4.0: the scheduler beyond 64 threads (§6), the TSV output's memory (3.5 M-precursor human library: 11.2 → 6.3 GiB, `docs/13-scaling-and-memory.md`), the user's own instrument files, the second command-line review (options without effect named in one warning; output-path collisions an error; `docs/cli-review-0.3/`), and the option names of 0.2.14 removed (Appendix A, `CHANGELOG.md`). V0.5.0: the fragment tolerance depends on the centroid's intensity (§2, §5), and the run model, what the calibration measured about the run along the gradient and by intensity, is kept in the output (`docs/15-run-model-plan.md`). **V0.3.0 changes no measurement here except timsTOF A01:** its traces at the defaults differ from V0.2.16's in 59,948 of 404,764 (DIA-NN's IDs; the cells V0.2.16 overwrote), with the agreement with DIA-NN unchanged to the fourth decimal; B01 and every Astral output are identical to V0.2.16's, and A01's output is now identical across repeats and thread counts (V0.2.16: 1-3 traces differed between identical runs). **V0.4.0 changes no result here:** re-run on everything, its comparisons with DIA-NN and OpenSWATH, the diaPASEF rows, the MS1 probes and the tolerance screens agree with V0.2.16's to the fourth decimal (with the A01 fix of V0.3.0 already in), its Astral traces at the benchmark's settings are bit-identical to V0.2's, and alternating V0.2.16 and V0.4.0 on the same node gave the same times (1 thread 12.6 s each, 64 threads 1.37–1.43 s; `results/v0.4.0-ab.log`). New is the scaling beyond 64 threads (§6). **V0.5.0 changes the traces at the defaults** through the intensity-dependent fragment tolerance: each centroid is accepted within the tolerance × the fragment mass-error width at its intensity / the overall width, as measured by the calibration, between 0.5 and the instrument's `mz_tol_intensity_max` (3; 1 for Orbitraps). On timsTOF the window widens for weak centroids and narrows for strong ones: fragment co-elution +10% (A01) and +8% (B01), fragments without signal at the apex 25% → 8–10% (§5); the agreement with DIA-NN's traces at the defaults falls, by construction, since DIA-NN's window is constant (§3). On Astral it only narrows for strong centroids: interference 4.7% → 3.6% at the same capture and shape (§4), traces otherwise as V0.4.0's (§3). Everything at fixed tolerances (the rule emulations, the MS1 probes, scheme B) reproduces V0.4.0 to the last digit shown; the calibration, the RT map and the RT window are unchanged. **Every cluster measurement was re-run with the V0.5.0 binary** (`results/v0.5.0/`, `scripts/cluster_bench_v0211.sh`, on `dax` on 2026-10-20); the V0.4.0 runs of 2026-10-19 (`results/v0.4.0/`) are cited where they are the comparison. The V0.4.0 report's tolerance screens had reused V0.2.11's extractions (the screen keeps any extraction it finds, and the screens shared one directory); V0.5.0's are new (§5). The V0.4.0 measurement paragraph, for reference: **Every cluster measurement was re-run with the V0.4.0 binary** (`results/v0.4.0/`, `scripts/cluster_bench_v0211.sh`, on `dax` on 2026-10-19): the thread series, the full libraries, the Astral pair table with all four configurations of ours, the diaPASEF comparisons and costs, and the tolerance screens. For the benchmark's shared settings (Appendix C) its Astral traces are bit-identical to those of cb466f0 (V0.2), on which the galleries and the §4 rule emulation were computed, and its calibration reproduces the offsets and

widths used there, so those carry over unchanged. Results that moved since V0.2: the A-V0.1 and P rows of §3 (re-extracted with the current calibration, ±0.6 points), the diaPASEF defaults (the m/z-dependent MS1 model, V0.2.2, and the measured RT window, V0.2.15, 7σ from V0.2.16; §3), and the speed of the default settings (§6). The laptop series (§6) is V0.4.0's release binary (2026-10-20), next to V0.2.7's.

What it is. `DIATraceExtractor` is an OpenMS TOPP tool (C++23, OpenMS develop) that extracts DIA fragment and precursor-isotope traces (XICs) from a predicted spectral library and writes them as Parquet or TSV. It calibrates retention time and mass against the run itself, needs no identifications, and does no scoring. This report gathers what has been measured about the quality of its traces and its speed against DIA-NN 1.7.12 and OpenSWATH on two instrument classes, an Orbitrap Astral and a timsTOF SCP (diaPASEF), the design decisions those measurements led to, and what is still open. Details live in the dossier (`00-dossier.md`), the extraction description (`EXTRACTION.md`) and the backlog (`BACKLOG.md`).

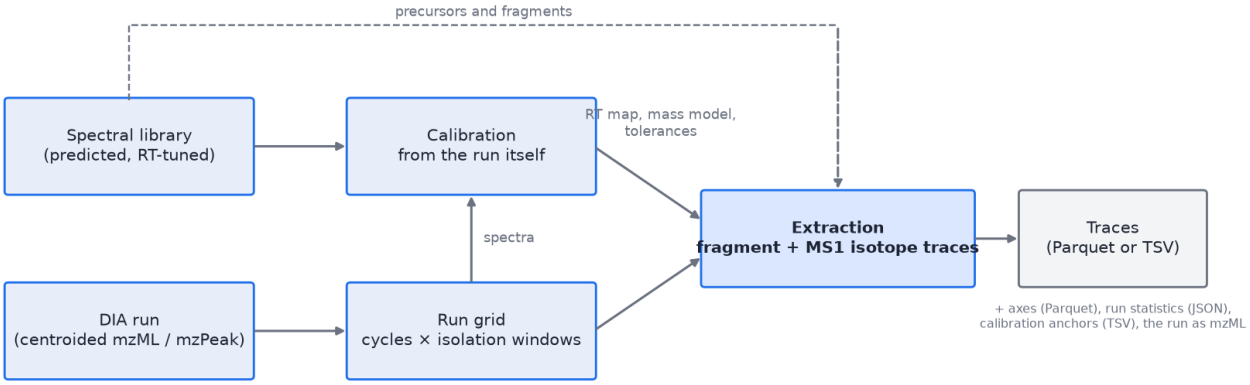
Verdict in one paragraph. At equal extraction rules the three tools produce the same traces, on both instruments; the differences between them are rule choices. Ours are the best-supported ones: a mass offset measured from the run (every data set here needs it, by 1–5 ppm, and the timsTOF MS1 needs an m/z-dependent one), tolerances expressed in multiples of the measured error width, and an MS1 window at the corrected m/z, which at DIA-NN's own width captures 98% of the true signal at 1% interference where DIA-NN's export captures 92% at 5% and OpenSWATH 100% at 22%. The extraction runs 19–28× faster than DIA-NN and 55–136× faster than OpenSWATH on the same run and library, in 2.7 GiB, with byte-identical output at 1 and 64 threads and, on run 01, from any of the three readers (fast mzML, OpenMS mzML, mzPeak); on a MacBook Pro (M4 Max) the same run takes 3.3 s at 16 cores, with OpenSWATH at 120 s. Two things are open and both concern the width of the window, not its centre: the tolerance multiple is set per instrument class from an identification screen rather than measured from the run, because the calibration anchors cannot see the interference that sets the optimum (V0.5.0 scales it per centroid by the measured width at its intensity, but the multiple itself is still the screen's); and the MS1 multiple of 4σ is on the narrow side (93% of the modelled true signal captured, §4) and should move to 5–6σ once re-screened.

1. Data, libraries, references

	Orbitrap Astral	**timsTOF SCP (diaPASEF)**
data set	ProteoBench DIA ion entrapment module: `LFQ_Astral_DIA_15min_50ng_Human_01–03` (PXD070049), 15 min, 50 ng, 5.8 GB mzML each	ProteoBench diaPASEF quantification module: conditions A and B, sample Alpha 01 (PXD062685), 25 ng; `.d` converted to mzPeak, and by this tool to mzML
library	DIALibGen (0.11.0, commit 6f26790; libraries named v0.11.1), human (3.5 M precursors) and entrapment (7.0 M); RT model tuned on DIA-NN's IDs of run 01 (`-rt_mode minutes`)	DIALibGen, HYE mixed species, 4.15 M precursors; predicted RT (`-rt_mode auto`)
reference traces	DIA-NN 1.7.12 (last open-source version, `--xic` export patched), OpenSwathWorkflow (OpenMS develop)	DIA-NN 1.7.12 on our mzML export (1.7.12 reads no `.d`)
restriction for comparisons	DIA-NN's 1% FDR IDs (78–93 k precursors)	DIA-NN's 1% FDR IDs (54,557 A01; 55,665 B01)

Sources, accessions and converters: Appendix B. DIA-NN exports traces only while searching, so its times include the search; OpenSWATH's include its scoring.

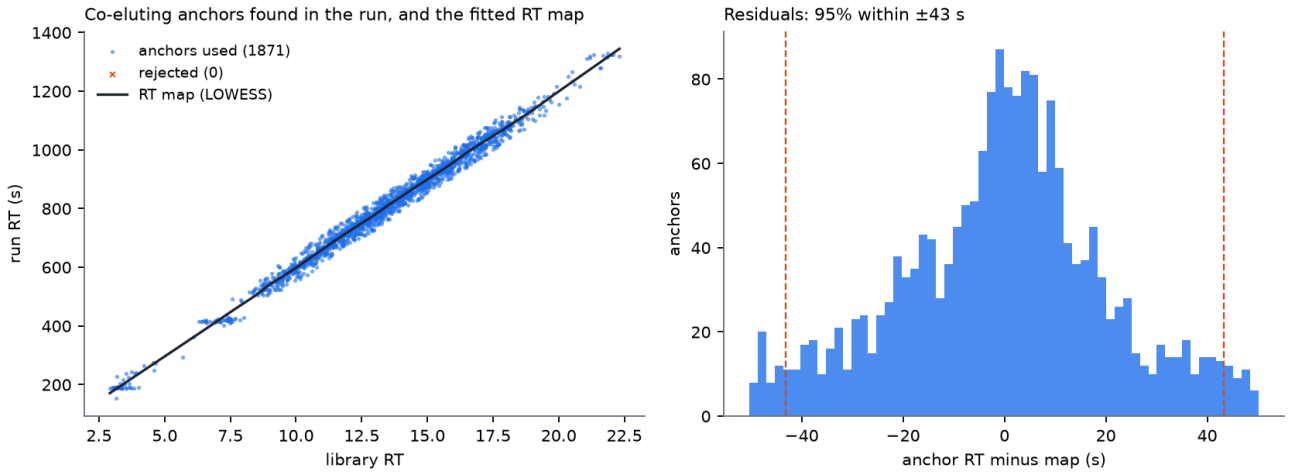
2. How the extraction works



- **Run grid.** Every centroid of the run goes into one memory-mapped pool (m/z as uint32 at 1e-5 Th, intensity as float32: 8 bytes per peak), indexed by isolation window and DIA cycle, with MS1 on its own scan axis. Three readers feed it: a fast mzML parser, the OpenMS mzML reader, and the mzpeak-openms library for mzPeak files. All three order tied centroids identically, so the traces are byte-identical whichever reader loaded the run.
- **Ion mobility (diaPASEF).** A diaPASEF MS2 frame is one spectrum that covers several isolation windows, each active over its own 1/K0 band, and holds the centroids of every TIMS scan, so one ion appears once per scan it spans. The reader uses the mobility in two steps and then integrates it out: each centroid is assigned to the isolation window whose mobility band contains it (the bands come from the frame's precursor descriptions), which restores the window selectivity a flat read of the frame would lose; then, within a window, the scans are collapsed: centroids within 15 ppm of each other are summed and placed at their intensity-weighted m/z. The grid then holds one ordinary spectrum per window and cycle and everything downstream is the same as on the Orbitrap. MS1 frames are collapsed over the whole mobility range in the same way. The library's predicted CCS column is not used: a precursor's trace integrates every mobility at which its fragments appear inside the window's band. The mzML export (`-out_mzml`) hands DIA-NN the same collapsed frames, so the comparisons in §3 are between extraction rules, not between mobility treatments. Mobility as an extraction axis (filtering a precursor's fragments to its predicted 1/K0, or mobility-resolved traces) is the next step for this instrument class and is not part of V0.2.
- **Calibration** (`EXTRACTION.md` §3). A panel of 5% of the library (every k-th row, never the `-ids` subset) is searched for co-eluting fragment apexes: a strict first pass over the whole run yields anchors for a robust LOWESS RT map; a relaxed second pass within ±max(4σ, 30 s) of the map refines it. The apex centroids of the strict anchors give the mass model: a median offset, or a line offset + slope × m/z when the m/z dependence is material (new in V0.2; it matters only on the timsTOF MS1, +15 ppm per 1000 Th; on Astral the fitted slopes are under 1 ppm per 1000 Th and change nothing measurable, `EXTRACTION.md`), and the error width as 1.4826 × MAD of the residuals. Tolerances are multiples of that width. Since V0.5.0 the calibration also measures the fragment width in up to ten intensity bands (from its first pass, at ±50 ppm, so that the narrowed search cannot shrink it), and the fragment tolerance of each centroid is scaled by the width at its intensity relative to the overall width (`-mz_tol_model intensity`; factors 0.5 to the instrument's `mz_tol_intensity_max`: 3, and 1 for Orbitraps, where only the narrowing helps, §4). On timsTOF A01 at 4σ that is ±22.5 ppm for the weakest tenth of centroids down to ±5.7 ppm for the strongest (constant: ±10.8 ppm).
- **Extraction.** Each precursor's fragments and MS1 isotopes are extracted within an RT window around the mapped RT: by default measured along the gradient, 7 × the robust RT scatter (1.4826 × MAD) of the anchors about the map plus 2 × their peak FWHM, per RT bin of about 150 anchors (V0.2.16; before, ±60 s), or fixed with `-rt_window`. The robust scatter is not biased by the calibration's RT search window, which cuts off the anchors' tails, and matched that of DIA-NN's identifications; but the residuals have heavy tails, so the multiple was chosen on all ten

benchmark configurations (Astral runs 01–03 with the tuned entrapment and human libraries, Astral run 01 with both predicted libraries, timsTOF A01/B01; `results/rtwin/all/`): as extracted, the windows contain 99.37–100% of DIA-NN's whole peaks (±60 s: 79–88% with predicted RT; 4 ×, chosen first on three runs, fell to 94.7% on the predicted human library, 6 × to 98.1%). They are about ±40–90 s with tuned libraries and ±250–300 s with predicted RT. The benchmark comparisons fix ±60 s. Extraction is a merge join of the sorted target windows against each spectrum's centroids, taking the most intense centroid per window (DIA-NN's rule, scheme A) or their sum (OpenSWATH's, scheme B).

- **Output.** Traces as one Parquet file (`-out`, one row per trace; 32 writers fill it in parallel, each appending its finished row groups, and one footer lists them in a fixed order) or as a long TSV (`-out_tsv`), with the axes file (`-out_axes`: global cycle axis, exact acquisition times per window, MS1 scan times), run statistics as JSON (`-out_stats`), the calibration anchors as TSV (`-out_calibration`, reusable as `-rt_anchors`), and the loaded run as a centroided mzML (`-out_mzml`, one spectrum per isolation window).
- **Scheduling.** One worker pool runs library reading, extraction, encoding and writing as task classes rebalanced by pipeline pressure, with in-order issue and an ordered commit, so the output does not depend on the thread count.



The two extraction schemes, A and B. The reference engines differ in a handful of rules, and the tool can follow either set, so that each comparison measures the rules and not the implementation:

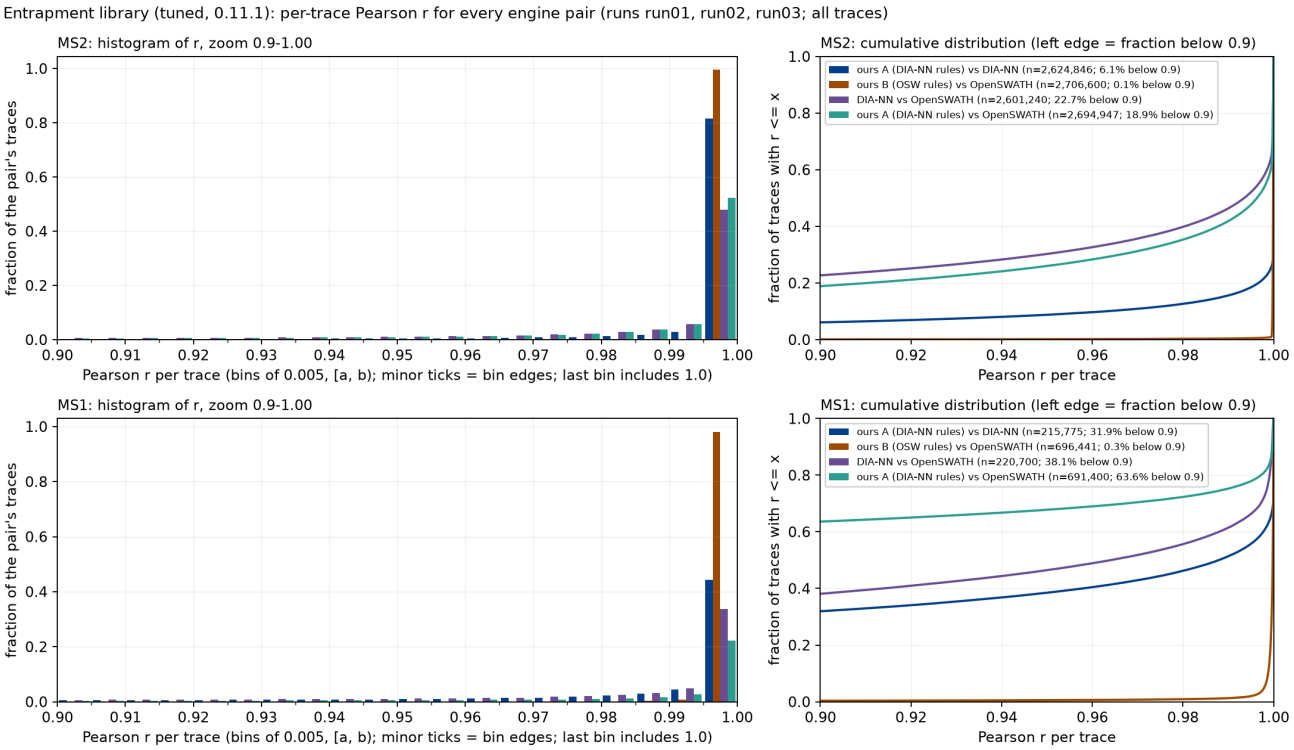
rule	A (DIA-NN's rules; the default)	B (OpenSWATH's rules)
value per spectrum	the most intense centroid in the m/z window	the sum of all centroids in the window; the whole column is one preset, `-scheme B`
mass correction	offsets (and the m/z-dependent term) measured from the run (`-mz_calibration auto`)	none: the window sits at the library m/z
tolerance	multiples of the measured widths (`-mz_accuracy_ms2`, `-mz_accuracy_ms1`), or fixed ppm; the fragment tolerance scaled per centroid by its intensity (V0.5.0)	DIA-NN's optimised MS2 accuracy for both levels, as the benchmark gave OpenSWATH (`-mz_tol_ms2`, `-mz_tol_ms1`)
MS1 isotopes	M+k at m/z + k · 1.00336 / z	M+k at m/z + k · 1.00336 regardless of charge
isolation windows	the window nearest the precursor	every window containing it (from V0.2.15; the same on the benchmark runs, which have no overlapping windows)
RT window, MS1 axis	±60 s around the mapped library RT; MS1 at its own scans	the same

Scheme B is a single preset (`-scheme B` , V0.2.1) because its three rules were easy to half-apply: once the measured mass correction became the default, a B command without `-mz_calibration off` extracted at the corrected m/z, and its parity with OpenSWATH fell from 95% to 64% of MS2 traces at r > 0.9 before this was caught (Appendix C). Three configurations of A appear below and are named throughout: **A-V0.2**, the defaults since V0.2 (MS2 9σ, MS1 4σ on Astral; in V0.5.0 with the intensity-dependent fragment tolerance, which on Astral only narrows the window for strong centroids); **A-V0.1**, the previous release's fixed rule (MS2 ±6 ppm, MS1 ±2.1 ppm around the measured constant offsets), on which the three-way comparison and the shape metrics of §3–§4 were run; and **P** (parity), A with both tolerances set to DIA-NN's optimised accuracy. Scheme B exists to show that the tool reproduces OpenSWATH's traces; its rules are not recommended (§4). Everything else is shared: the run grid, the RT map, the merge join, the output.

3. Are the traces the same? (both instruments, DIA-NN's IDs)

Astral, run 01, shared tuned library, all 79,953 DIA-NN IDs, every library fragment (`results/v0.5.0/astral/cmpA.tsv` , `cmpB.tsv` , `cmpAV01.tsv` , `cmpP.tsv` ; V0.5.0: B, P and A-V0.1 identical to V0.4.0's, A-V0.2 at the defaults the same to the digits shown, its strong centroids now in a narrower window):

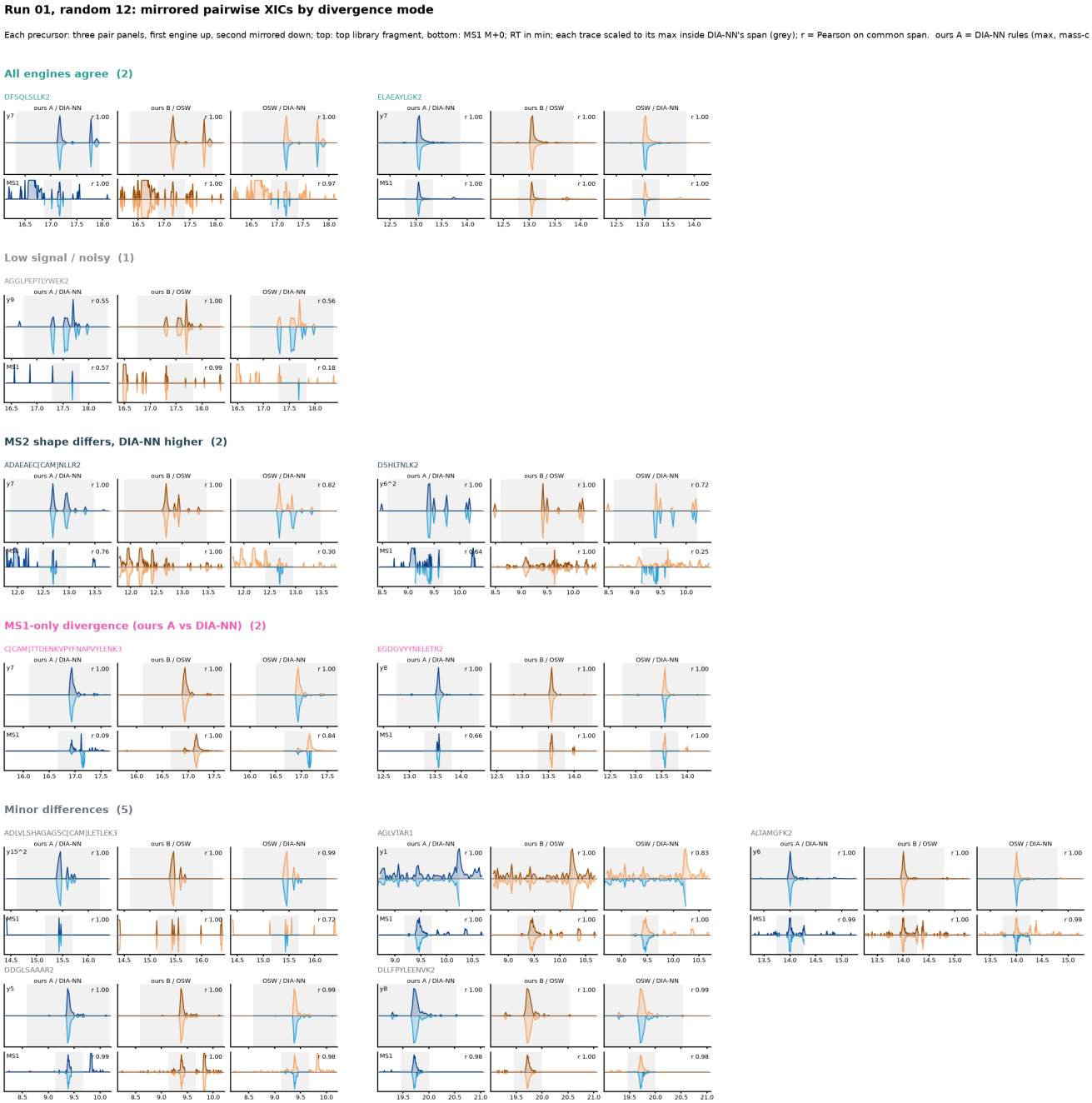
pair	MS2 median r	MS2 r > 0.9	MS1 M+0 median r
ours B vs OpenSWATH	0.9999	95.3%	0.9995
ours P (DIA-NN's tolerances) vs DIA-NN	1.0000	91.4%	0.954
ours A-V0.1 vs DIA-NN	1.0000	91.6%	0.979
ours A-V0.2 (defaults) vs DIA-NN	0.9997	83.1%	0.977
DIA-NN vs OpenSWATH	0.971	62.7%	0.953



(The figure pools runs 01–03 with an earlier configuration, both tolerances at DIA-NN's accuracy and MS1 on the cycle axis, dossier §2; the shape of the distributions is the point.) Ours B reproduces OpenSWATH nearly exactly; ours A reproduces DIA-NN at MS2. The two reference engines disagree with each other more than either disagrees with its matching configuration of ours. The remaining MS2 tail of P against DIA-NN (9% of traces below r = 0.9 in the table; 3.5% of the traces non-zero in both engines, dossier §3.2) is entirely its float32 quadratic window centre, shifted by about 0.5 ppm from ours: emulating that arithmetic reproduces DIA-NN's traces in a 3,000-precursor sample with

none of 34,923 traces differing at r < 0.99 or L1 > 0.01 (dossier §3.2). It is a DIA-NN rule choice, and its quadratic describes the true mass error worse than a constant does. At the defaults, A-V0.2's wider MS2 window (±7.9 ppm against DIA-NN's ±5.9) lowers the share of traces at r > 0.9 to 83%, by construction of the comparison; §4 shows what the wider window does to the traces themselves.

A random sample of the traces, all four arms. The figure shows 12 of 50 randomly drawn precursors (seed 20261010) as mirrored pairs: ours A-V0.2 against DIA-NN, ours B against OpenSWATH, and OpenSWATH against DIA-NN, the top library fragment above and MS1 M+0 below, each trace scaled to its maximum inside DIA-NN's exported span (grey). The full 50 are in `figures/gallery_v02/trace_mirror_random.png`, the same precursors overlaid in `trace_gallery_by_mode.png`, and one figure per mode in `trace_gallery_mode_*.png` (`scripts/trace_gallery_features.py`, `trace_gallery.py`, `trace_gallery_mirror.py`).



Each of the 73,350 precursors with traces in all four arms is assigned one divergence mode from the per-trace metrics (first matching rule wins; thresholds anchored on the B-vs-OpenSWATH re-implementation noise floor):

mode	share	in the random 50
all engines agree	19.0%	9
minor differences	27.1%	16
MS1 differs, ours A vs DIA-NN (MS2 agrees)	25.6%	10
MS2 shape differs, DIA-NN higher	11.5%	9
MS2 shape differs, OpenSWATH higher	3.6%	1
DIA-NN's XIC span off-centre (truncated common span)	6.2%	3
low signal / noisy	5.9%	2
scale only (sum vs max)	1.2%	0

No mode is caused by an RT shift: all engines read the same spectra. The "MS2 shape differs" modes are the OpenSWATH window centre (§4) and DIA-NN's quadratic centre; the "MS1 differs" mode is DIA-NN's MS1 export at the uncorrected m/z against our corrected, narrow window (§4), visible in the figure as DIA-NN's MS1 trace carrying extra peaks ours does not. The galleries were regenerated for this release (the 2026-10-10 versions in `figures/` show the earlier configuration of the correlation figure).

timsTOF diaPASEF, runs A01 and B01, DIA-NN 1.7.12 on our mzML export, DIA-NN's IDs, every library fragment (`results/v0.5.0/diapasef/cmp_*.tsv`; V0.5.0, whose default RT window is measured along the gradient, ±130–357 / ±122–306 s here instead of ±60 s; V0.2.15's 4σ window, ±75–207 / ±71–178 s, gave the same rows to the rounding shown). Calibration finds 775 / 710 anchors in these 25 ng samples (342–374 used for the mass model), MS2 width 2.7 ppm; the MS1 error carries an m/z slope of +13–15 ppm per 1000 Th, so the defaults use the m/z-dependent model, MS1 width 2.2 ppm (3.2 ppm constant):

ours vs DIA-NN	MS2 median r	MS2 r > 0.9	MS1 r > 0.9
A01, defaults (MS2 4σ = ±10.8 ppm scaled by intensity: ±22.5 … ±5.7; MS1 ±8.9 ppm, m/z-dependent)	0.904	50.9%	60%
A01, V0.4.0's defaults (`-mz_tol_model constant`: MS2 ±10.8 ppm)	0.991	87.0%	60%
A01, defaults with the constant MS1 model (MS1 ±12.9 ppm)	0.904	50.9%	66%
A01, at DIA-NN's ±11.8 ppm (constant)	0.9990	92.6%	64%
B01, defaults (MS2 4σ = ±10.6 ppm scaled: ±25.9 … ±5.7; MS1 ±8.7 ppm)	0.903	50.7%	62%
B01, V0.4.0's defaults (constant)	0.973	77.1%	62%
B01, at ±11.8 ppm (DIA-NN: ±12.6; constant)	0.994	90.1%	67%

(With V0.2.11's ±60 s window the same rows read 86.0 / 86.0 / 91.9 / 76.0 / 89.2% at MS2 and 57 / 63 / 61 / 59 / 64% at MS1, `results/v0.2.11/diapasef/`: the measured window adds the traces whose apex lay beyond ±60 s.)

At equal tolerance the engines agree on diaPASEF as on Astral (the rows at DIA-NN's ±11.8 ppm are V0.4.0's). At the defaults V0.5.0's traces differ from DIA-NN's in half the traces, by construction: DIA-NN's window is constant, ours admits weak centroids up to about twice as far and strong ones only half as far; those traces co-elute better (§5, +10%) and lose far fewer fragments at the apex (25% → 10%). V0.4.0's constant window, slightly narrower than DIA-NN's, cost a few points of agreement for the same reason. The MS1 agreement of 60–66% looked like a defect and is not: where both engines have signal the values are identical, and each has zeros where the other has signal, so the two windows sit at different centres. Placing our MS1 window at the uncorrected library m/z with DIA-NN's MS2 accuracy raises the agreement to 81%, and widening that uncorrected window to ±20 ppm drops it to 46% (`cmp_A01_ms1raw11.8.tsv`, `cmp_A01_ms1raw20.tsv`, top 6 fragments): DIA-NN's export rule of §4, visible here because the MS1 offset is 3.5 ppm plus the m/z slope instead of Astral's 1.3

ppm. The m/z-dependent model moves our MS1 centre further from the uncorrected one, which is why it lowers the agreement from 66% to 60%. Our MS1 traces are the mass-corrected ones.

4. Which rules are right? (re-extraction from the raw centroids, 3,000 IDs)

How this was assessed. There is no ground truth for a trace, so the rules were judged on the centroids themselves. For a random sample of 3,000 DIA-NN-identified precursors of Astral run 01, every centroid within ±25 ppm of each fragment and MS1 isotope target was recorded at every point of the trace (`scripts/quality_reextract.py`, reading the mzML directly). A mass-error model was fitted to the apex centroids of the top-6 fragments of the confident IDs: a mean μ(m/z, intensity), quadratic in m/z with an upward drift for strong centroids, and a width σ(intensity) = √(0.83² + 70²/I) ppm, i.e. 2.4 ppm at 10^3 and 0.86 ppm at 10^5. A centroid is counted as true signal when it lies within 3σ(I) of μ. Each engine's rule (window centre, width, max or sum) is then applied to the recorded centroids; **capture** is the share of the true signal in the peak region (apex ±4 cycles) that the rule returns, **interference** the share of what it returns that is not true signal (an upper bound, since the tails of the error distribution hold real signal too). Shape metrics are computed on the same re-extracted traces: **co-elution** is the mean pairwise Pearson r of a precursor's fragment traces over ±4 cycles around the apex, **cosine at apex** the cosine between the square-rooted apex intensities and the library intensities, **dropouts** the share of fragments without signal at the apex. The rule emulation was validated first: for every engine it reproduces the engine's own exported traces with a median r of 1.0 (`scripts/quality_sample.py`, `scripts/quality_v02_rules.py`, `results/analysis/quality/`; details in `analysis/trace_quality.md`).

MS2 (DIA-NN's accuracy ±5.9 ppm; ours A at the V0.2 defaults, 9σ = ±7.9 ppm around the measured offset with its m/z term):

rule	capture	interference	co-elution	cosine at apex	dropouts
ours A (V0.2 defaults, max, ±7.9 ppm)	**99.9%**	4.7%	0.521	0.920	14.8%
DIA-NN (quadratic centre, max, ±5.9 ppm)	99.5%	2.8%	0.522	0.919	14.9%
ours B = OpenSWATH (no correction, sum, ±5.9 ppm)	97.3%	3.1%	0.467	0.903	16.9%
ours A V0.5 (defaults: ±7.9 ppm × the intensity factor, 0.56–1)	99.9%	**3.6%**	0.522	0.920	14.7%
ours A V0.5, factors not capped at 1 (0.56–1.88)	99.99%	5.3%	0.492	0.921	15.7%

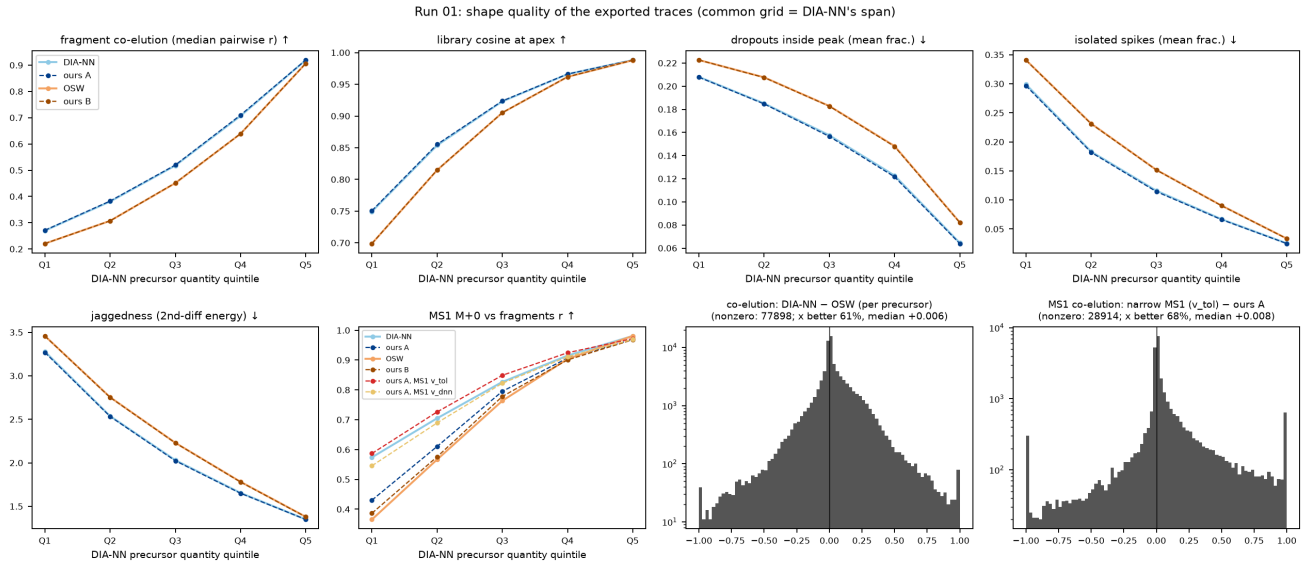
(V0.5 rows: `scripts/quality_v05_rule.py`, `results/analysis/quality/v05_run01_engine_rules.tsv`, with V0.5.0's calibration of the run; the other rows as before.) V0.5.0's intensity-dependent tolerance, capped at 1 on Orbitraps (`mz_tol_intensity_max`), narrows the window for strong centroids only: a quarter less interference at the same capture and shapes. Letting it widen for weak centroids as well (the factors as measured) captures the last 0.1% but admits more interference and lowers the co-elution, which is why the cap is 1 here; on timsTOF the widening is what helps (§5). The truth model's width is itself intensity-dependent, so it favours an intensity-dependent window to some degree; the comparison of the two V0.5 rows, which differ only in the weak centroids, does not depend on that.

Ours A and DIA-NN give the same trace shapes; A's wider window trades 1.9 points of interference for the last 0.4% of signal (at ±5.9 ppm around our offset it is 99.6% / 2.9%, indistinguishable from DIA-NN). B's loss is its centre: with no correction, the ±5.9 ppm window ends only 2.7 ppm beyond the true −3.2 ppm offset. Max versus sum is irrelevant on Astral MS2: at most 4×10^{-5} of the points hold two centroids in one window.

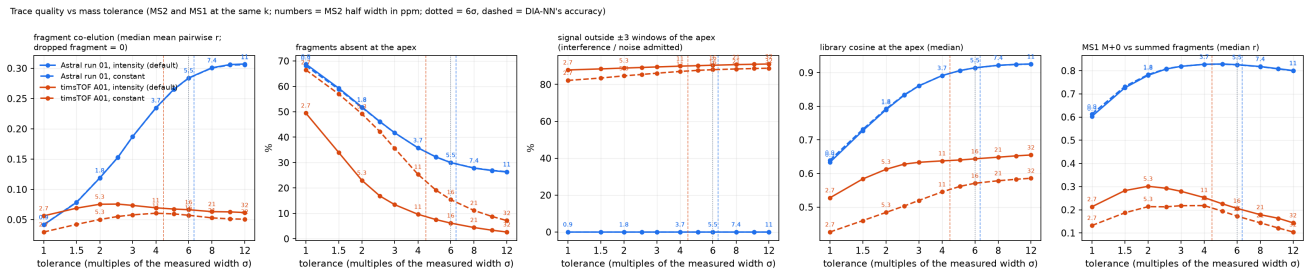
MS1, M+0 (peak region apex ±6 MS1 scans; truth model 3σ(I) around the MS1 median, σ 0.85 ppm):

rule	capture	interference
ours A (V0.2 defaults: measured offset with m/z term, max, 4σ = ±1.43 ppm)	92.6%	**0.0%**
ours A at DIA-NN's width (measured offset, ±2.13 ppm)	**98.0%**	0.9%
DIA-NN export (uncorrected m/z, max, ±2.13 ppm)	91.6%	5.4%
ours B = OpenSWATH (no correction, sum, ±5.9 ppm)	100%	22%

The corrected centre is what matters: at DIA-NN's own width it captures 98% at 0.9% interference where DIA-NN's uncorrected export captures 92% at 5.4%, and the wide OpenSWATH window admits a quarter of its peak intensity as interference. The 4σ default is on the narrow side by this truth model: the width measured on the anchors (0.36 ppm) comes from strong apex centroids, while weak MS1 centroids scatter more (the intensity dependence of the errors, `EXTRACTION.md`), so ±1.43 ppm leaves 7% of the modelled true signal outside. At ±2.1 ppm around our offset (A-V0.1), MS1-to-fragment co-elution on the exports ranks ours (0.886) above DIA-NN (0.880) and OpenSWATH (0.826); the 4σ export has not been shape-scored. A 5–6σ MS1 default is the change this points at. OpenSWATH additionally places isotope k at m/z + k·1.00336 without dividing by the charge, so its M+1 and M+2 traces of charge ≥ 2 precursors are a different ion.



5. Trace quality as a function of the tolerance (both instruments)



The DIA-NN-identified precursors of each run extracted at k × σ, k = 1 … 12, and scored on the traces (`scripts/tolerance_screen.py`, V0.5.0, `results/v0.5.0/tolscreen/`; σ fixed at the screen's values, Astral 0.92 / 0.40 ppm, timsTOF 2.66 / 3.38 ppm for MS2 / MS1; window centres and RT windows from the current calibration), with V0.5.0's intensity-dependent fragment tolerance (solid) and with the constant one (`-mz_tol_model constant`, dashed; V0.4.0's rule). Medians over the precursors (the co-elution means in `docs/figures/tolerance_screen_*.tsv`). The V0.4.0 report's screens had reused V0.2.11's extractions (±60 s windows, an older calibration), so their absolute values differ from these:

	MS2 co-elution optimum, constant	intensity (default)	at the default multiple, constant → intensity	DIA-NN's own accuracy
Astral run 01	k = 10–12 (0.306)	k = 10–12 (0.307)	9σ: equal (0.300–0.306 → 0.301–0.306); dropouts 27.9 → 27.8%	6.4σ (±5.9 ppm)
timsTOF A01	k = 4 (0.0606)	**k = 2.5 (0.0757)**	4σ: 0.0606 → 0.0695 (+15%); dropouts 25.4 → 9.7%; cosine 0.545 → 0.638	4.4σ (±11.8 ppm)

On Astral, where the intensity factor is capped at 1 (it only narrows the window for strong centroids, §4), the two rules give the same curves; the MS2 curve saturates (heavy error tail, dropouts keep falling). On timsTOF the intensity-dependent window lifts the whole curve: at every k it co-elutes better, loses far fewer fragments at the apex and matches the library better, at the cost of more off-peak signal (87.0 → 89.9% at 4σ; the measured RT window is wide here, so most of every trace is off-peak), and its optimum moves to 2.5–3σ. A lower timsTOF multiple would gain more, but the calibration narrows its mass search with the same multiple, and 3σ narrowed the measured RT window below its coverage target on one configuration (`docs/15-run-model-plan.md` §9), so the multiples stay 9σ (Astral) and 4σ (timsTOF). MS1 co-elution (MS1 at the same k, against the summed fragments) rises with the better fragment traces on timsTOF (0.218 → 0.253 at 4σ). The timsTOF MS1 width has dropped from 3.38 to 2.2 ppm with the m/z-dependent model, so the MS1 multiple of this screen's σ is not the default's (§4).

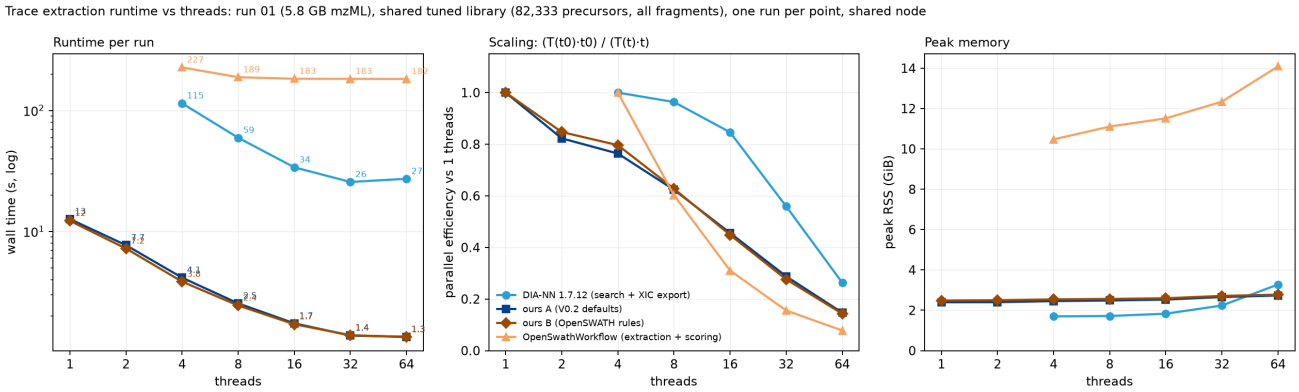
Can the calibration find the multiple by itself? Not as a rule. The anchors' own co-elution curve does peak at the Astral optimum (k = 8), but every anchor-based criterion tried rises monotonically to 8σ on timsTOF too: the all-fragment co-elution curve of the anchors, the same on the weaker half of the anchors, on their weak fragments only, and on both; and the anchor yield at a 1% decoy-calibrated false rate, which the shifted-fragment decoys make meaningless (0–5 decoys per run). Anchors are present precursors whose fragments stay correlated as the window widens; the screen's optimum comes from interference in weak and absent precursors, a population the calibration does not have. **Decision:** the multiples are set per instrument class from the run's metadata (MS2 9σ Orbitrap/Astral, 4σ timsTOF and other TOF, 6σ with a warning when unknown; MS1 4σ), as explicit options (`-mz_accuracy_ms2`, `-mz_accuracy_ms1`; `-mz_tol_ms2`/`-mz_tol_ms1` for fixed ppm).

Peptide decoys do not supply that population either (V0.2.16, `results/decoys/`). Libraries with pseudo-reversed decoys (DIALibGen `-generation:decoys pseudo_reverse`) let the calibration score the library's own decoys, which stand for absent peptides. The anchor yield at a 1% decoy-calibrated false rate saturates at 4–5σ on Astral run 01 (1,338 anchors at 4σ, 1,343 at 6σ, 1,336 at 8σ), far from the screen's 8–10σ, and is unstable on timsTOF A01 (54–295 anchors pass). As an anchor filter the decoys are not robust either: at 1% only 157 timsTOF anchors remain; at 5% the timsTOF MS2 width widens from 2.8 to 3.6 ppm; on Astral the cleaner anchors narrow the mass width (1.01 → 0.78–0.87 ppm) but understate the RT scatter, so the RT windows hold 96–97% of DIA-NN's peaks instead of 99.7%. The fixed criteria and the instrument-class multiples stay; the decoy path remains as an experiment (`ODIA_ANCHOR_FDR`).

6. Performance

Machine: 2 × AMD EPYC 9654 (`dax`: 192 cores, SMT, 2.3 TB), shared node at a load of 9–50 (≤ 0.13 per hardware thread) during the V0.4.0 runs (a first pass at a load of 90–130, from other jobs, was 10–50% slower and was repeated; `results/v0.4.0/threads_loaded/`); inputs on local `/scratch`; one run per point unless stated; peak RSS from `/usr/bin/time`.

Runtime vs threads, Astral run 01, shared tuned library (82,333 precursors, all fragments); ours V0.5.0 (`results/v0.5.0/threads/`; the benchmark's fixed ±60 s window), DIA-NN and OpenSWATH from the cb466f0 series with unchanged binaries (`results/threads_cb466f0/`):



threads	ours A (defaults)	ours B (OpenSWATH rules)	DIA-NN 1.7.12	OpenSwathWorkflow
1	12.7 s / 2.4 GiB	12.2 s / 2.5 GiB	–	–
2	7.1 s	6.7 s	–	–
4	4.2 s / 2.4 GiB	3.8 s / 2.5 GiB	115 s / 1.7 GiB	227 s / 10.5 GiB
8	2.5 s	2.4 s	59 s	189 s
16	1.7 s	1.6 s	34 s	183 s
32	1.3 s	1.3 s	26 s	183 s
64	1.3 s / 2.7 GiB	1.2 s / 2.8 GiB	27 s / 3.3 GiB	182 s / 14.1 GiB

Ours is 28–30× faster than DIA-NN at 4 threads and 19–20× from 16 threads up, and 55–137× faster than OpenSWATH. Ours flattens beyond 32 threads at this library size: the tool's own stages take 0.7 s at 64 threads (ingest 0.25 s, calibration 0.05 s, extraction 0.36 s; tool total 0.99 s), the remaining 0.4 s is process start-up and teardown; B, which skips the calibration, is 0.04 s faster. OpenSWATH stops scaling at 8 threads while its memory grows to 14 GiB. Ours read the inputs from local disk, the references from the network file system (Appendix C); the references were not run below 4 threads. Against the cb466f0 series the V0.2.7 ingest changes are within the run-to-run spread on Linux (64 threads: 1.44 s for cb466f0, 1.28 s for V0.2.7 at a load of about 50, 1.35 s for V0.2.11, 1.37 s for V0.2.15 with its single output file, 1.45 s for V0.2.16, 1.34 s for V0.4.0 and 1.32 s for V0.5.0), as expected: libstdc++ never had the small-read problem described for the laptop below.

Full libraries, Astral run 01, 64 threads, top 6 fragments, the defaults of V0.5.0 (`-rt_mode minutes`; RT window measured, ±45–78 s for the human and ±99–185 s for the entrapment library; `results/v0.5.0/full/`):

library	tool total (wall incl. start-up), 2 runs	ingest	calibration	extraction	peak RSS	traces
human, 3.49 M precursors	5.8–5.9 s (6.2–6.3 s)	0.3 s	0.4 s	3.6–3.7 s	3.9 GiB	31.3 M
entrapment, 7.0 M precursors	12.3–12.5 s (12.9–13.0 s)	0.3 s	0.9 s	9.2–9.3 s	5.2 GiB	63.0 M

As V0.4.0 (5.8 s, 12.5–12.6 s): on Astral the intensity-dependent tolerance only narrows windows, the output grows by 0.4% (entrapment library, 32 threads: 3,796 → 3,812 MB).

The extraction follows the measured window, which with 7σ is about ±60 s for the human library and wider for the entrapment library; V0.2.15's 4σ window (±27–48 / ±58–107 s) took 4.8 s and 9.6–10.2 s. V0.2.11 had measured 5.0–5.1 s and 8.1–8.4 s with ±60 s and a directory of 32 part files; V0.2.7 had measured 5.7–5.8 s and 8.0–9.3 s at a load of 45–55, cb466f0 5.6–5.9 s and 9.1 s at a load of about 21, and 6d26c49 8.6–9.0 s on the entrapment library on a quieter node (dossier §4). The thread table above uses `/usr/bin/time` wall clock, this one the tool's own total.

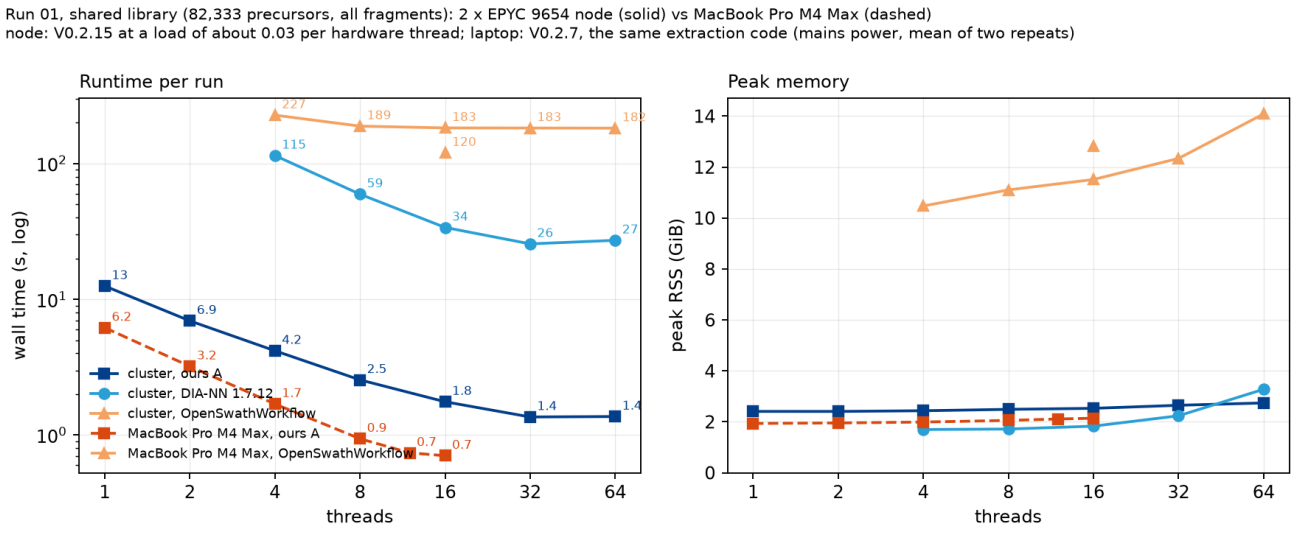
Beyond 64 threads (the entrapment library at the defaults, two repeats; `results/v0.5.0/scaling/`; V0.4.0: `results/v0.4.0/scaling/`; before: `results/scaling64/base/` with the V0.3.0 scheduler; `docs/13-scaling-and-memory.md`):

threads	V0.5.0	V0.4.0	V0.3.0
64	12.3–12.5 s / 5.2 GiB	12.4 s / 5.2 GiB	11.9 s / 5.3 GiB
96	10.4–10.7 s / 5.8–5.9 GiB	10.2–10.6 s / 5.7 GiB	10.5–10.7 s
192 (dax's physical cores, the default)	11.6–11.7 s / 5.9–6.0 GiB	11.8–12.3 s / 5.9 GiB	21.9–22.2 s / 5.7 GiB
384	12.5–12.6 s / 6.1 GiB	12.7–12.8 s / 6.1 GiB	40.6 s / 6.2 GiB

Up to V0.3.0 every finished task woke all idle workers of the scheduler, which queued for its lock and slept again (3.6 M context switches at 192 threads), and at most 64 units ran at once; V0.4.0 wakes one worker per state change and lets the units in flight grow with the threads. The default (the physical cores) is now as fast as 64 threads; 96 threads remain the fastest point on this machine, where memory bandwidth across the two sockets and the 32 Parquet writers set the limit. The output is identical at every thread count.

diaPASEF (A01 and B01, 4.15 M-precursor library, 32 threads, V0.5.0, `results/v0.5.0/diapasef/odia_*_all.log`): 70 / 64 s (A01 / B01; V0.4.0: 64 / 52 s), 15.5–15.6 GiB peak RSS, 36.7 M traces; the mzPeak per-spectrum read and the mobility collapse take 26–28 s of it (the part that varies most between runs; V0.4.0: 19–25 s), the calibration 4 s, the extraction 32–36 s (V0.4.0: 28–33 s): the weak centroids' wider window holds more centroids, and the output grows by 15% (A01: 22.4 → 25.6 GB, as fewer points are zero). The extraction grows with the measured RT window (±130–357 / ±122–306 s; V0.2.15's 4σ window, ±75–207 / ±71–178 s, took 56 / 51 s with 21–23 s of extraction); with V0.2.11's ±60 s (80%) both runs took 44 s (extraction 12 s, `results/v0.2.11/diapasef/`). The peak falls in the ingest, while the calibration panel is extracted concurrently, and varies between runs (13.5–16.5 GiB over the archived runs).

On a laptop (MacBook Pro, Apple M4 Max: 12 performance + 4 efficiency cores, no SMT, 128 GiB, macOS 26.6, inputs on the internal SSD; Appendix C; not re-run for V0.5.0, whose speed on the node is V0.4.0's). Same run and shared library, mean of two repeats on mains power: the V0.4.0 release installed with Homebrew (2026-10-20, `results/macos-benchmark/v0.4.0/`) and, in parentheses, the V0.2.7 binary built on the laptop (318e5aa, `results/macos-benchmark/v0.2.7/`); repeats within 2% except 1 thread (10%):



threads	ours A, V0.4.0 (V0.2.7)	ours B, V0.4.0 (V0.2.7)	OpenSwathWorkflow
1	6.3 s (6.2 s) / 2.0 GiB	5.8 s (5.9 s) / 2.1 GiB	–
2	3.2 s (3.2 s)	3.0 s (3.1 s)	–
4	1.7 s (1.7 s)	1.6 s (1.6 s)	–
8	0.95 s (0.94 s)	0.90 s (0.89 s)	–
12 (performance cores)	0.74 s (0.74 s)	0.68 s (0.69 s)	–
16	0.68 s (0.70 s) / 2.2 GiB	0.65 s (0.66 s) / 2.3 GiB	120 s / 12.8 GiB (2026-10-13)

The release binary (signed, its libraries bundled) is as fast as the one built on the laptop at every thread count. (A V0.3.0 series on battery, 2026-10-19, `results/macos-benchmark/v0.3.0/`, was up to 25% slower in its first repeats.) The full 7.0 M-precursor library takes 20.6–21.3 s at 16 threads in 4.5 GiB with the measured RT window (±99–185 s; extraction 16.3 s at 14 active cores), against 14.2–15.3 s in 4.0 GiB with V0.2.7's fixed ±60 s: the wider window extracts two to three times as many cycles, as on the node (§6 full libraries). Per core the laptop is the faster machine, and in absolute terms it matches the node: 0.68 s at 16 cores against 1.3 s at 64 threads on the node (OpenSWATH: 120 s against 183 s); ours A speeds up 9.3× (V0.2.7: 8.9×) from 1 to 16 threads.

The laptop's V0.2.3 series (3.3 s at 16 threads, 41 s at 1 thread, 19–23 s of system time) exposed what V0.2.7 fixes (`12-macos-benchmark-note.md`): libc++'s `ifstream::read` refills a 4 KiB buffer with one `read()` per 4 KiB, and macOS serialises concurrent small reads of one file in the kernel, so the two passes over the 5.8 GB mzML (~2.9 M syscalls) cost 21 s of system time at 16 threads and kept the ingest at about 3 s regardless of the thread count (libstdc++ reads large requests directly, so the node never showed it); and the ordered chunk pipeline's look-ahead of 32 chunks kept the workers waiting on the in-order commit 85 % of the time, on Linux as well. Both passes now use positional reads (`platform::FileReader`), and the fast parser's look-ahead is unbounded (a parked chunk holds only metadata; +40 MB peak RSS). Ingest at 16 laptop threads 2.9 → 0.27 s, 1 thread 41 → 6.2 s overall; traces and axes byte-identical to V0.2.3 at 1 and 16 threads. The laptop had also exposed three defects, all present on Linux too; the first two were fixed on the laptop (tag V0.2.3), the third on the node (V0.2.4):

- `--help` aborted (an unregistered TOPP subsection, since `calibration:mz_model` became a visible option on 2026-10-12; `--helphelp` had aborted on every build);
- every run aborted after writing its output, because OpenMP workers hand their thread-local buffer caches back to a pool that a static destructor had already torn down; glibc tolerates the dead mutex, libc++ throws. The pool is now never destroyed;
- `-threads 1` fell back to the serial OpenMS mzML parser instead of the chunk parser with one worker: 70 s instead of 14 s on the node for the shared library (`results/threads_cb466f0/t1/odiaA_serial_parser.log` vs `odiaA.log`); 41 s instead of 6.2 s on the laptop.

What the speed rests on beyond the design of §2 (dossier §4, `06-parallel-efficiency.md`, `07-contention-diagnosis.md`): the parsers decode straight into the peak pool; a bucket index over m/z skips gaps in the merge join; trace and Arrow buffers are recycled with a per-thread cache; 32 Parquet parts are written by their own tasks. The library-load transient (decoded row groups, parts and the joined copy coexisting) and the mzPeak reader's serialised decode (scales to 10–14 cores) are the known remaining costs.

7. Reproduction

- Code: https://github.com/okohlbacher/dia-trace-extract-prototype, release tag `V0.4.0` (this report is attached to the release); build instructions in Appendix A, data sources in Appendix B, command lines in Appendix C.

- Benchmarks: `scripts/benchmark_refined.sh` (three-way comparison on a shared recalibrated library), `scripts/bench_threads.sh` (thread series), `scripts/bench_macos.sh` (laptop), `scripts/plot_machine_scaling.py`, `scripts/compare_traces.py` (pairwise metrics), `scripts/quality_reextract.py`, `quality_sample.py` and `quality_v02_rules.py` (§4), `scripts/tolerance_screen.py` and `plot_tolerance_screen.py` (§5), `scripts/plot_thread_scaling.py` (§6), `scripts/doc_figures.py` (§2 figures). Mass-error dumps: `ODIA_DUMP_MASS_ERRORS=file`.
- Inputs: Appendix B; libraries in `library/` (DIALibGen 0.11.0 + 6f26790, `library/generate.sh`, `library/hye_timstof.log`).
- Tests: `ctest` in the build directory (7 tests: TraML extraction, OpenSWATH parity, byte-level Parquet compare).

Appendix A: repository and installation

Repository. https://github.com/okohlbacher/dia-trace-extract-prototype, tag `V0.5.0`. History: V0.2 (663dc39) the extraction as benchmarked; V0.2.1 the `-scheme` preset; V0.2.2 macOS portability; V0.2.3 the `--help` and exit-time fixes (laptop branch); V0.2.4 the one-thread ingest fix and the laptop results; V0.2.5 the report's version stamp; V0.2.6 release packages for Linux, macOS and Windows (platform layer, release CI); V0.2.7 the mzML ingest with positional reads and an unbounded in-order commit, and the re-measured §6; V0.2.8 release builds for five platforms; V0.2.9 self-contained Windows packages (x86_64, arm64); V0.2.10 a relocatable macOS package; V0.2.11 the first macOS package signed with the project's Developer ID and notarized, and every cluster measurement re-run; V0.2.12 continuous integration on pushes and pull requests, checksums and provenance attestations for the release files; V0.2.13 the disk image's stapling retried, this report published by the workflow; V0.2.14 the macOS package runs when installed through Homebrew (library validation off, as Homebrew re-signs the bundled libraries; the build's absolute rpaths removed, which had loaded Homebrew's libomp next to the bundled one), checked in CI; V0.2.15 the command line reviewed (adversarially, with two other models): the RT map fitted from the run, all cores up to 64 and an RT window measured along the gradient by default, a warning for a wrong RT scale, one Parquet file per run, consistent option names, 13 basic options, parity switches folded into `-scheme B`, and every cluster measurement re-run. V0.2.16 the physical cores by default, the RT-window multiple from all benchmark data (7σ), the 0.2.14 option names accepted again until 0.4, the peptide-decoy experiment, a macOS `.pkg` built and tested (published once signed), the Homebrew tap updated by the workflow (once its deploy key is set). V0.3.0 the instrument defaults as data files (`-instrument`, `-instrument_file`), value sources in the output, a shorter log, the TOPPBase patch (one-line unknown-option message, `-write_ini` on the current names), and the fix of the merge join for RT windows that shrink faster than the RT grows. V0.4.0 the scheduler beyond 64 threads, the TSV output's memory, the user's own instrument files, the second command-line review and the 0.2.14 option names removed. V0.5.0 the intensity-dependent fragment tolerance by default and the run model in the output. The mzPeak reader is the submodule `external/mzpeak` (okohlbacher/mzpeak-openms, tag `partitioned-read-2026-10-11`); the library generator is okohlbacher/DIALibGen (0.11.0, commit 6f26790 on branch `compact-parquet-plain-strings`, which also carries the build fix for current OpenMS develop).

Binary releases. Every tag is built, tested (ctest plus an extraction from the installed folder in a clean environment) and packaged by `.github/workflows/release.yml` against the OpenMS commit pinned in `ci/openms.lock`; pushes to main and pull requests run the same on Linux x86_64. Each release lists the files' checksums in `SHA256SUMS` and carries a build provenance attestation per file (`gh attestation verify FILE -R okohlbacher/dia-trace-extract-prototype`). The GitHub release carries self-contained folders with the tool, the libraries it links and OpenMS's shared data:

platform	package	run
Linux x86_64, aarch64 (glibc ≥ 2.39)	`DIATraceExtractor-0.5.0-linux-<arch>.tar.gz`	`bin/DIATraceExtractor`
macOS Apple silicon	`…-macos-arm64.tar.gz`, `.dmg`	`bin/DIATraceExtractor`
Windows x86_64, arm64	`…-windows-<arch>.zip`	`bin\DIATraceExtractor.exe`

```
brew tap okohlbacher/odia && brew install diatraceextractor      # macOS; the tap tracks the
latest release
```

The macOS package is signed with the Developer ID (hardened runtime), notarized, and the `.dmg` stapled; from a tarball or Homebrew the first start can take minutes while macOS checks each bundled library online once (a tarball cannot carry the ticket). The Windows packages bundle the Visual C++ and OpenMP runtimes but are not code-signed yet, so SmartScreen warns on first start.

Building from source. Requirements. Linux, GCC ≥ 13, CMake ≥ 3.24, Ninja; an OpenMS develop (≥ 3.7) build tree with its vcpkg dependencies, which provide Arrow/Parquet and, with `boost-json` and `boost-program-options` added to the OpenMS vcpkg manifest, the Boost ≥ 1.89 the mzPeak library needs (another `BoostConfig.cmake` directory can be given with `-DMZPEAK_BOOST_DIR`), and from V0.3.0 yaml-cpp (also in that manifest) and one patch to OpenMS's `TOPPBase` (`patches/openms-toppbase-deprecated-options.patch`, applied with `git -C OpenMS apply`; the release workflow applies it). Python 3 with pyarrow, pandas, numpy and matplotlib (`tools/pyenv` here) for the scripts; `markdown` and `weasyprint` for the PDF.

```
git clone https://github.com/okohlbacher/dia-trace-extract-prototype.git && cd dia-trace-
extract-prototype
git checkout V0.5.0
git submodule update --init
# OpenMS develop, built with its vcpkg manifest (here at OpenMS/OpenMS-build via `cmake --
preset linux-x64-release`)
O=$PWD/OpenMS/OpenMS-build
cmake -S . -B build -G Ninja -DCMAKE_BUILD_TYPE=Release -DCMAKE_PREFIX_PATH="$O;$O/
vcpkg_installed/x64-linux-dynamic"
cmake --build build -j 16
ctest --test-dir build        # 17 tests: TraML extraction, OpenSWATH parity, determinism, RT-
window spans, the command line
```

macOS (Apple silicon) from source. Supported since V0.2.2 (the Linux-only calls are guarded). `scripts/bench_macos.sh` builds OpenMS develop (its `macos-arm64-release` preset, Homebrew libomp; 1–3 h the first time) and the tool, fetches the inputs and runs the benchmark, one phase at a time or `all`; the ctest suite passes on macOS (7/7).

Use.

```
# the defaults: RT map, RT window, mass offsets and tolerances measured from the run, all
physical cores
DIATraceExtractor -in run.mzML -lib library.parquet -out traces.parquet
# a library tuned to this run (RT in run minutes): skip fitting the RT map; a JSON summary
DIATraceExtractor -in run.mzpeak -lib library.parquet -rt_mode minutes -out traces.parquet -
out_stats stats.json
DIATraceExtractor -in run.mzpeak -out_mzml run.mzML      # the loaded run as centroided mzML,
for other tools
DIATraceExtractor -in run.mzML -lib library.parquet -instrument Astral -out traces.parquet   #
metadata without the instrument
DIATraceExtractor --help                                 # the 14 everyday options; --helphelp:
all
```

From V0.2.15 the defaults need no tuning: `-rt_mode auto` (before: library RT taken as run seconds unless `-rt_mode` was given), the physical cores unless `-threads` is given (V0.2.16; before: TOPP's 1; an INI file sets the count only when it has a `threads` entry; beyond about 64 threads the extraction does not speed up on this node: 7 M precursors on 2 × EPYC 9654 took 9.4 s at 64 threads, 22 s at 192, 43 s at 384), the measured RT window (§2), and `-out` as one Parquet file (before: a directory of 32 part files). The names follow one pattern (`-rt_*`, `-mz_*_ms2`/`_ms1`, `-ms1_*`, `-out_*`): `-mz_tol` became `-mz_tol_ms2`, `-mz_sigma_ms2`/`_ms1` `-mz_accuracy_ms2`/`_ms1`, `-mass_calibration` `-mz_calibration`, `-mz_offset` `-mz_offset_ms2`, `-calibration:mass_model` `-calibration:mz_model`, `-ms1_isotopes -1` the flag `-no_ms1`, `-out_shards` `-write_parts`. Options that only served the

benchmark's parity experiments were folded into `-scheme` or removed (`-aggregation`, `-ms1_isotope_spacing`, `-window_assignment`, the linear RT map, 21 calibration thresholds). V0.2.16–V0.3.x accepted the names of 0.2.14 (translated, with one `WARNING:` line); V0.4.0 removed them: an old name is an error preceded by one line naming the replacement, and an INI file of 0.2.14 by one saying so (convert it with 0.3: `-write_ini new.ini -ini old.ini`).

From V0.3.0 the instrument-dependent defaults are data files (`share/DIATraceExtractor/instruments/*.yaml`): `default.yaml` (fragment tolerance 6 × the measured width, MS1 4 ×, RT window 7σ + 2 FWHM), `orbitrap.yaml` and `astral.yaml` (9), `tof.yaml` and `timstof.yaml` (4), each value with its source; they reproduce the class rule of V0.2.16. `-instrument auto` (default) reads the instrument from the run's metadata; `-instrument <name>` names it, `-instrument_file` adds a file of one's own (strictly validated). The Parquet footer and `-out_stats` record each value used with its source (`given`, `instrument-default:Astral`, `default`, `measured`). The log has one short line per stage (detail with `-debug 1`), every warning starts with `WARNING:`, a mistyped option gets one line with the closest name, and `-write_ini` writes the current names only (`-write_ini new.ini -ini old.ini` converts an old INI).

Input formats: mzML (indexed, centroided) and mzPeak; libraries as DIALibGen compact Parquet, DIA-NN/OpenSWATH TSV, TraML or PQP.

Appendix B: data sets

Orbitrap Astral (ProteoBench DIA ion entrapment module). Three single-species human runs of the "LFQ Benchmark Dataset - Generation Beta" (Van Puyvelde et al., Dhaenens lab, Ghent; ProteomeXchange **PXD070049**, https://www.ebi.ac.uk/pride/archive/projects/PXD070049): `LFQ_Astral_DIA_15min_50ng_Human_01/02/03.raw`, Orbitrap Astral, 15-min gradient, 50 ng. ProteoBench mirrors them with the module's search database:

file	source
`LFQ_Astral_DIA_15min_50ng_Human_01.raw` (3.1 GB), `_02.raw`, `_03.raw`	https://proteobench.cubimed.rub.de/raw DIA-astral-entrapment/ (also `all_data_Entrapment_DIA_Astral.ta` 5.6 GB)
`ProteoBenchFASTA_Entrapment_Human_with_contaminants_entrapment_pep.zip` (2,841,736 peptide-level entries: human targets, contaminants and entrapment peptides, 1:1)	https://proteobench.cubimed.rub.de/fas
`UP000005640_9606.fasta` (UniProt human reference proteome, 20,652 entries; the human library)	https://www.uniprot.org/proteomes/UP000005640

The `.raw` files were converted to centroided, indexed mzML with OpenMS `FileConverter` from the same OpenMS develop build (Thermo raw read through its ThermoRawFileParser bridge, vendor centroiding; `benchmark-data/DIA-astral-entrapment/mzML/convert_*.log`); mzPeak variants were made with mzpeak-convert 0.17.2 / 0.18.0 and with this tool's `MzMLToMzPeak`.

timsTOF SCP diaPASEF (ProteoBench DIA quantification, precursor-ion level, diaPASEF module). Three-species mixture (human, yeast, E. coli commercial digests in the proportions of Van Puyvelde et al., 2022, Scientific Data 9:126, https://www.nature.com/articles/s41597-022-01216-6; expected log2 A/B: human 0, yeast −1, E. coli +2), 25 ng in triplicate, Bruker timsTOF SCP with an UltiMate 3000 RS nanoLC and a 25 cm IonOpticks Aurora column, diaPASEF with precursor m/z 400–1000, 1/K0 0.64–1.37, 0.96 s cycle. ProteomeXchange **PXD062685** (announced by the module; at the time of writing the files are served by ProteoBench):

file	source
`ttSCP_diaPASEF_Condition_A_Sample_Alpha_01_11494.d.zip` (7.3 GB), `..._Condition_B_Sample_Alpha_01_11496.d.zip` (7.1 GB); replicates 02/03 of both conditions available, not used	https://proteobench.cubimed.rub.de/raws/diaPASEF/
`ProteoBenchFASTA_MixedSpecies_HYE.zip` (31,889 entries: three proteomes plus contaminants prefixed `Cont_`, Frankenfield et al., J. Proteome Res. 2022)	https://proteobench.cubimed.rub.de/fasta/ProteoBenchFASTA_MixedSpecies_HYE.zip

The `.d` folders were converted with mzpeak-convert 0.18.0 (`benchmark-data/diaPASEF/convert_*.log`); the mzML files DIA-NN read are this tool's `-out_mzml` export of the loaded grid.

Reference software. DIA-NN 1.7.12 (https://github.com/vdemichev/DiaNN, release 1.7.12, the last open-source version; built here with a one-line fix to its `--xic` export, `tools/DiaNN-1.7.12/`); OpenSwathWorkflow from the OpenMS develop build used for the tool itself; DIALibGen 0.11.0 + 6f26790 (https://github.com/okohlbacher/DIALibGen) with AlphaPeptDeep ONNX models for the predicted libraries.

Appendix C: command lines

All extractions of ours in this report were run with the V0.5.0 binary (`bin-releases/DIATraceExtractor-0.5.0`, sha256 4142cbe3160c; the timsTOF rows and the Astral A-V0.1 and P rows with the build before `mz_tol_intensity_max`, f7089939b218, whose traces there are the same: the cap is 3 for TOF instruments and these rows use fixed constant tolerances) by `scripts/cluster_bench_v0211.sh` and `results/v0.5.0-bench2.sh` on `dax` (results in `results/v0.5.0/`; `STEPS` selects: 1 thread series, 2 full libraries, 3 Astral pair table, 4 diaPASEF, 5 Astral A-V0.1 and P, 6 tolerance screens). The reference engines were run once and their traces are reused (provenance at the end). Paths: `$S` = `/scratch/$USER/odia-bench` (inputs staged from `benchmark-data/` and `library/`), `$X` = the binary, `$DIANN` = `tools/DiaNN-1.7.12/diann-1.7.12-xicfix`, `$OSW` = `OpenMS/OpenMS-build/bin/OpenSwathWorkflow`, `$PY` = `tools/pyenv/bin/python`; `$M` = the Astral run 01 mzML, `$L` = its shared library (`proteobench_entrapment_astral_nce25_v0.11.1_tuned_run01_ids_run01.parquet`).

Option names, as in the tool: `-rt_*` retention time, `-mz_*` m/z with `_ms2`/`_ms1` for the level, `-ms1_*` MS1 traces, `-out_*` output files. Every value not given below is the default (RT map, RT window, mass offsets and tolerances measured from the run). Where the commands fix `-rt_window 60` and `-rt_mode minutes`, they reproduce the benchmark's shared settings; there the traces are bit-identical to those of the V0.2 binary (cb466f0) on which the galleries and the rule emulation of §4 were computed (`scripts/same_traces.py`, `results/v0.4.0/astral/same_as_cb466f0.txt`; V0.5.0's ours B is identical to them, ours A differs in its intensities from V0.5.0's tolerance, `results/v0.5.0/astral/same_as_cb466f0.txt`).

Astral pair table and galleries (§3, §4). The four configurations of ours on run 01 and the comparison with DIA-NN's and OpenSWATH's traces of the same run, library and precursors:

```
 $X -in $M -lib $L -rt_mode minutes -rt_window 60 -top_fragments 0 -threads 32 -out
odiaA.parquet                 # A-V0.2 (the defaults)
$X -in $M -lib $L -rt_mode minutes -rt_window 60 -top_fragments 0 -mz_tol_ms2 6 -mz_tol_ms1
2.114775 \
   -calibration:mz_model constant -threads 32 -out
odiaAV01.parquet                                          # A-V0.1 (V0.1's tolerances)
$X -in $M -lib $L -rt_mode minutes -rt_window 60 -top_fragments 0 -mz_tol_ms2 5.9051 -
mz_tol_ms1 5.9051 -threads 32 -out odiaP.parquet   # P
$X -in $M -lib $L -rt_mode minutes -rt_window 60 -top_fragments 0 -mz_tol_ms2 5.9051 -
mz_tol_ms1 5.9051 -scheme B -threads 32 -out odiaB.parquet   # B
$PY scripts/compare_traces.py --odia odiaA.parquet --axes odiaA.axes.parquet --diann results/
gallery_v02/diann/diann_xic.parquet \
    --openswath results/gallery_v02/osw_traces.parquet --out
cmpA.tsv                                        # likewise for AV01, P, B
$PY scripts/trace_gallery_features.py results/gallery_v02 results/analysis/gallery_v02/
```

```
features_run01.parquet
$PY scripts/trace_gallery.py results/gallery_v02 results/analysis/gallery_v02/
features_run01.parquet docs/figures/gallery_v02 results/analysis/gallery_v02/gallery_run01
$PY scripts/trace_gallery_mirror.py results/gallery_v02 results/analysis/gallery_v02/
gallery_run01_selection.tsv docs/figures/gallery_v02 --sample random
```

timsTOF diaPASEF (§3, §6). `.d` to mzPeak with mzpeak-convert 0.18.0; DIA-NN reads this tool's mzML export of the loaded run; B01 likewise with the B-condition files. All precursors (costs, §6), then DIA-NN's IDs at the defaults, with the constant MS1 model, at DIA-NN's accuracy, and the MS1 window probes:

```
 /ceph/ibmi/abi/oliver/bin/mzpeak-convert ttSCP_diaPASEF_Condition_A_Sample_Alpha_01_11494.d -o
A01.mzpeak
$X -in A01.mzpeak -out_mzml A01.mzML -threads 32                    # frames split by window,
TIMS scans collapsed within 15 ppm
$DIANN --f A01.mzML --lib library/diann17/hye_timstof.tsv --out report.tsv --temp temp --
threads 32 --verbose 1 --qvalue 0.01 --xic 60
$X -in A01.mzpeak -lib library/hye_timstof.parquet -threads 32 -out all.parquet -out_stats
all.json -out_calibration anchors.tsv
I="-in A01.mzpeak -lib library/hye_timstof.parquet -ids results/diapasef/diann_A01_ids.txt -
top_fragments 0 -threads 32"
$X $I -out idsdef.parquet                                           # the defaults
$X $I -calibration:mz_model constant -out idsconst.parquet          # constant MS1 model
$X $I -mz_tol_ms2 11.8 -mz_tol_ms1 11.8 -out idsP.parquet           # DIA-NN's accuracy
$X $I -mz_tol_ms2 11.8 -mz_tol_ms1 7 -out ms1_7.parquet             # MS1 window probes: 4, 7,
20, 30 ppm around our offset
$X -in A01.mzpeak -lib library/hye_timstof.parquet -ids results/diapasef/diann_A01_ids.txt -
threads 32 \
   -mz_calibration off -mz_offset_ms2 -4.6 -mz_tol_ms2 10.8 -mz_offset_ms1 0 -mz_tol_ms1 11.8 -
out ms1raw11.8.parquet   # uncorrected MS1 centre (also +-20 ppm)
$PY scripts/compare_traces.py --odia idsdef.parquet --axes idsdef.axes.parquet --diann results/
diapasef/diann_A01_xic.parquet --out cmp_A01_idsdef.tsv
```

RT window (§2, Appendix A). The measured window was chosen on the anchors and DIA-NN's peaks of Astral run 01 and timsTOF A01/B01 (`results/rtwin/` : `run.sh` , `analyse.py` , `sigma.py` , `rule.py` ; end to end with the tool's own output: `validate.sh` , `coverage.py`).

Tolerance screen (§5). One extraction per multiple k with `-mz_tol_ms2 k*sigma_ms2 -mz_tol_ms1 k*sigma_ms1 -top_fragments 0 -ids ...` , scored on the traces:

```
 $PY scripts/tolerance_screen.py --extractor $X --run $M --lib library/refined/
proteobench_entrapment_astral_nce25_v0.11.1_shared_run01.parquet \
    --ids results/tolscreen/astral_run01_ids.txt --sigma-ms2 0.92 --sigma-ms1 0.40 --rt_mode
minutes --out astral_run01
$PY scripts/tolerance_screen.py --extractor $X --run $S/input/diapasef/A01.mzpeak --lib
library/hye_timstof.parquet \
    --ids results/diapasef/diann_A01_ids.txt --sigma-ms2 2.66 --sigma-ms1 3.38 --rt_mode auto
--out timstof_A01
$PY scripts/plot_tolerance_screen.py docs/figures/tolerance_screen "Astral run
01=astral_run01.tsv:6.4" "timsTOF A01=timstof_A01.tsv:4.4"
```

Speed (§6). The thread series (`scripts/bench_threads.sh` , ours only; DIA-NN and OpenSWATH from the cb466f0 series below) and the full libraries, `$t` = 1 ... 64:

```
 $X -in $M -lib $L -rt_mode minutes -rt_window 60 -top_fragments 0 -threads $t -out
odiaA.parquet
$X -in $M -lib $L -rt_mode minutes -rt_window 60 -top_fragments 0 -mz_tol_ms2 5.9051 -
mz_tol_ms1 5.9051 -scheme B -threads $t -out odiaB.parquet
$X -in $M -lib $S/input/human_UP000005640_astral_nce25_v0.11.1_tuned_run01.parquet -rt_mode
minutes -threads 64 -out full_human.parquet
```

```
$X -in $M -lib $S/input/proteobench_entrapment_astral_nce25_v0.11.1_tuned_run01.parquet -
rt_mode minutes -threads 64 -out full_entrap.parquet
```

The laptop series (`scripts/bench_macos.sh bench`; `OMP_WAIT_POLICY=PASSIVE`; machine: `docs/12-macos-benchmark-note.md`) ran the thread-series commands with `$t` in 1 2 4 8 12 16 and the full library at 16 threads, on mains power: V0.4.0 as released through Homebrew (`X=$(brew --prefix)/bin/DIATraceExtractor bash bench_macos.sh bench`; `results/macos-benchmark/v0.4.0/`, `docs/figures/laptop_m4max_v040_summary.tsv`; V0.3.0 on battery: `v0.3.0/`, `docs/figures/laptop_m4max_v030_summary.tsv`) and V0.2.7 = 318e5aa built on the laptop (`results/macos-benchmark/v0.2.7/`, `docs/figures/laptop_m4max_summary.tsv`). The OpenSwathWorkflow run is from 2026-10-13 (`RUN_OSW=1`).

Reference engines (run once, `scripts/benchmark_refined.sh`; Astral in `results/refined/proteobench_entrapment_astral_nce25_v0.11.1_tuned_run01/run01/` and `results/gallery_v02/`, the thread series in `results/threads_cb466f0/` and `results/threads/`). The libraries in each engine's format come from `scripts/lib_to_diann17_tsv.py` (`--rt_scale 1`) and `scripts/lib_to_openswath_tsv.py` (`--rt_scale 60`); DIA-NN's XICs are converted with `scripts/diann17_xic_to_parquet.py`, OpenSWATH's with `TextExporter -xic:mode explode`.

```
# DIA-NN 1.7.12: round 1 on the tuned library gives the IDs that restrict the shared library,
round 2 exports the XICs
$DIANN --f $M --lib library/diann17/proteobench_entrapment_astral_nce25_v0.11.1_tuned_run01.tsv
--out report.tsv --temp temp --threads 64 --verbose 1 --qvalue 0.01
$DIANN --f $M --lib lib_diann17.tsv --out report.tsv --temp temp --threads 64 --verbose 1 --
qvalue 0.01 --xic 60
# OpenSwathWorkflow, full window = 2 x DIA-NN's accuracy
$OSW -in $M -tr lib_openswath.tsv -out_chrom osw.xic -out_features osw.featureXML -
rt_extraction_window 120 \
     -mz_extraction_window 11.8102 -mz_extraction_window_unit ppm -mz_extraction_window_ms1
11.8102 -mz_extraction_window_ms1_unit ppm \
     -ms1_isotopes 2 -Calibration:auto_irt:enabled false -threads 64
```

Libraries. `library/generate.sh human|entrapment` (DIALibGen, Astral NCE 25); the timsTOF library as recorded in `library/hye_timstof.log`; the tuned RT models with `DIALibGen -mode tune` on DIA-NN's run-01 IDs (`library/*.log`); the `_ids_runNN` and `_shared_runNN` libraries with `scripts/filter_library.py` on DIA-NN's report (`scripts/benchmark_refined.sh`, step 2).

Provenance of earlier results. The galleries and the §4 rule emulation were computed on the cb466f0 (V0.2) outputs in `results/gallery_v02/` (traces identical to V0.4.0's, above); the mass-error dumps of the mass model on 4e89acc (`ODIA_DUMP_MASS_ERRORS=file`); the earlier series are kept for comparison in `results/v0.2.11/` (V0.2.11), `results/threads_v0.2.7_d963cb5/` and `results/full_v0.2.7_318e5aa/` (V0.2.7), `results/threads_cb466f0/` and `results/full_cb466f0/` (V0.2), and `results/tolscreen/` (4934ea5).